A Stochastic-Computing Tensor Unit and an RV32I Core on a 9K-LUT FPGA: Bit-Exact Co-Design for Tiny-MLP Inference

[TBD:authors]
[TBD:affiliation]

***Abstract*—Stochastic computing (SC) replaces a multiplier by a single logic gate acting on random bit-streams, which makes it attractive for the smallest FPGAs, where dedicated multipliers are scarce. We present the co-design of an SC tensor processing unit (SC-TPU) and a multi-cycle RV32I processor for an 8640-LUT4 Gowin GW1NR-9 device (Sipeed Tang Nano 9K). The SC-TPU is a 16×8 weight-stationary tile of bipolar XNOR multipliers with accumulative parallel counters, driven by 8-step leap-forward LFSRs in three decorrelation modes. Its behaviour, including the integer post-processing, is specified bit-exactly so that a Python golden model, the training flow and the RTL must agree bit for bit. For a 64-16-10 MLP on the 8×8 digits set, simulation of the golden model gives 95.0 % test accuracy for the expected-value (exact integer) datapath and 94.4 % (95 % CI 91.6–96.4 %) for SC with independent generators at stream length 256 (rev A seeds); shared random numbers drop to 65.6 %. An analytical variance model predicts the measured RMS stochastic error within a factor 0.74–1.13. The CPU passes 44 ISS-checked programs under four bus timing configurations (176/176 runs) and closes timing at 53.26 MHz (worst of three place-and-route seeds) against a 27 MHz clock. All results are simulations with an open-source flow; no hardware has been operated. SC-TPU implementation figures are pending ([TBD:tpu-lut], [TBD:soc-fmax-worst]).**

***Index Terms*—stochastic computing, neural network accelerator, RISC-V, FPGA, LFSR, bit-exact golden model, open-source FPGA toolchain**

I. Introduction

Low-cost FPGAs with fewer than ten thousand look-up tables are widely used in education, prototyping and small embedded products, but they offer only a handful of hard multipliers. The GW1NR-9 device used here has 8640 LUT4 positions, 6480 flip-flops, 26 block RAMs and 20 18×18 multipliers [1]. A binary int8 neural-network datapath on such a device is limited to at most 40 hard 9×9 multiply–accumulate (MAC) lanes, or must spend roughly 70 LUTs per soft 8×8 multiplier (rough CALCULATION).

Stochastic computing [2], [3] encodes a number as the probability of a one in a random bit-stream. A multiplication then reduces to a single AND (unipolar) or XNOR (bipolar) gate, and the precision becomes a run-time parameter: doubling the stream length L halves the variance of the result. Neural networks tolerate moderate arithmetic noise, which has made them a prominent application of SC [4]–[6]. The price is latency (L clock cycles per product), the cost of random-number generation, and a sensitivity to correlation between streams.

A second, less discussed difficulty is *verification*: an SC result is a random variable, so a hardware/software comparison is meaningless unless the random sequence is defined as precisely as the arithmetic. This paper treats the random-number generators, their seeding and their sequencing as part of a bit-exact architectural contract.

Contributions.

1) A complete, small SoC architecture for the Tang Nano 9K: a multi-cycle RV32I CPU [7] orchestrating a 16×8 weight-stationary bipolar SC tile with accumulative parallel counters (APC), using only an open-source flow [8]–[10] (Section III).
2) A bit-exact specification of the SC datapath, including an 8-step leap-forward LFSR, three RNG decorrelation modes and integer post-processing, mirrored by a Python golden model used for quantisation-aware training with SC noise (Section IV).
3) An accuracy study versus stream length and RNG mode with 95 % confidence intervals and a seed ensemble, and an analytical variance model checked against simulation (Section V).
4) A verification methodology with ISS-based differential testing of the CPU (44 programs × 4 bus timings) and fault-injection checks of the testbenches, and an explicit account of what remains unverified (Section VI).

II. Background and Related Work

A. Stochastic number encoding

A stochastic number (SN) is a bit-stream $b_0, \ldots, b_{L-1}$. In *unipolar* encoding it represents $p = P(b_k{=}1) \in [0, 1]$; in *bipolar* encoding it represents $v = 2p - 1 \in [-1, 1]$, which admits signed weights with a single stream per operand [2], [3]. A stochastic number generator (SNG) converts a binary code c into a stream by comparing it with a pseudo-random number r: $b = [r < c]$. With r uniform on $\{0, \ldots, 2^n - 1\}$, $P(b{=}1) = c/2^n$.

B. Multiplication and accumulation

For two independent unipolar streams the AND gate gives $P(a \wedge b) = p_a p_b$. For bipolar streams the XNOR gate gives

$$P\big(\overline{a \oplus b} = 1\big) = \tfrac{1}{2}\,(1 + v_a v_b)\,, \tag{1}$$

i.e. the bipolar product [3], [4]. Accumulating many products with a multiplexer scales the sum down by the fan-in and loses

TABLE I
MEMORY MAP (SPEC REV A, SOURCE: ARCHITECTURE SPECIFICATION)

Base	Size	Content
0x0000_0000	4 KiB	RAM (program + data), reset PC = 0
0x1000_0000	32 B	LED, UART_TX/STATUS, CYCLE, TOHOST
0x2000_0000	4 KiB	SC-TPU: registers 0x000–0x017, INPUT 0x040, SCORE 0x080, HIDDEN 0x0C0, BIAS 0x100, weight RAM 0x800–0xFFF
other	–	read 0, write ignored, sticky bus_err

precision. An accumulative parallel counter instead counts the ones of all product streams in each cycle and adds the count to a binary accumulator, so the sum is exact in expectation and leaves the stochastic domain; SC-DCNN [6] studied such APC-based inner products for deep convolutional networks. Kim *et al.* [5] exploited the stream length as a run-time energy–accuracy knob in deep networks.

C. Precision, correlation and LFSRs

An L-bit estimate of p has variance $p(1-p)/L$, so every factor of four in L adds one bit of precision. Equation (1) holds only for independent streams; streams generated from the same or from shifted random sequences are correlated and bias or inflate the error [3]. Linear-feedback shift registers (LFSRs) are the usual cheap random source. Taking n consecutive bits of a single-step LFSR as r yields successive values that are shifted copies of each other, and two LFSRs with the same polynomial produce lagged copies of the same maximal-length sequence. Both effects matter in a small design that cannot afford many independent generators.

D. Small RISC-V cores and open FPGA flows

PicoRV32 [11] is a size-optimised RV32 core with the same native valid/ready memory interface that we adopt. The Yosys and nextpnr tools [8], [9], together with the reverse-engineered Gowin database of Project Apicula [10], provide a fully open flow for the GW1N family; we use it exclusively.

III. ARCHITECTURE

A. SoC overview

Fig. 1 shows the SoC. A single 27 MHz clock domain is driven by the board oscillator (no PLL); the reset button passes through a two-flip-flop synchroniser into a synchronous active-high reset. The CPU is the only bus master. Slaves are selected by address bits [31:28] (Table I); an access to an unmapped region returns zero, completes normally and sets a sticky debug flag, so a software error can never hang the bus. The bus is a native valid/ready protocol: the master holds mem_valid, address, data and byte strobes stable until the slave returns a one-cycle mem_ready pulse.

B. RV32I multi-cycle CPU

The CPU implements the RV32I base instruction set [7] with FENCE as a no-op; there are no CSRs and no interrupts. ECALL, EBREAK, illegal encodings (strictly decoded, including all CSR instructions, unknown funct3/funct7 values,

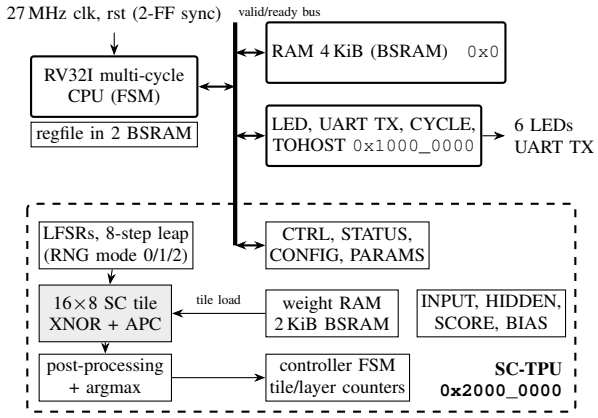


Fig. 1. SoC block diagram (architecture specification rev A). The CPU is the only bus master; the SC-TPU is a memory-mapped slave whose weight RAM is time-shared between CPU writes and tile loads.

0x00000000 and 0xFFFFFFFF) and misaligned accesses or jump targets halt the core with one of six trap causes. Because the CPU only orchestrates the accelerator, it is optimised for area:

- a state machine with the states CLEAR, FETCH, DECODE, EXECUTE, SHIFT or MEM, and WB; CLEAR zeroes the 32 registers once after reset (32 cycles);
- the register file is held in two block RAMs (one per read port), costing no LUT or flip-flop for its 1 kbit of state;
- a single 33-bit adder serves arithmetic, comparisons and address generation; branch targets are computed in DECODE so that the comparison in EXECUTE can reuse it;
- shifts are iterative (4 or 1 bit per cycle) instead of a barrel shifter.

The resulting cycles per instruction at zero wait states are (CALCULATION from the FSM): 3 for LUI, AUIPC, JAL, FENCE and branches, 4 for ALU operations, JALR and stores, 5 for loads, and $4 + \lfloor n/4 \rfloor + (n \bmod 4)$ for a shift by n.

C. SC-TPU dataflow

The workload is a 64-16-10 multilayer perceptron with a clipped-ReLU hidden layer and an argmax output. Every input, activation and weight is an 8-bit code c representing the bipolar value $v(c) = (c - 128)/128 \in [-1, 127/128]$. The array is a weight-stationary tile of $N_L = 16$ neuron lanes by $N_I = 8$ input lanes (Fig. 2). $N_L = 16$ equals the hidden width, so one pass of a layer-1 tile serves all hidden neurons. Layer 1 is processed in $T = 64/8 = 8$ tiles and layer 2 in $T = 16/8 = 2$ tiles; the six unused neuron lanes of layer 2 are padding and are ignored by the argmax. A fully parallel layer-1 array (1024 lanes, 8192 weight flip-flops) does not fit the 6480 flip-flops of the device, which fixed the tile choice.

For one tile the 128 weights W_{ji} and the 8 input codes x_i are loaded into registers; then, during $L = 2^K$ stream cycles

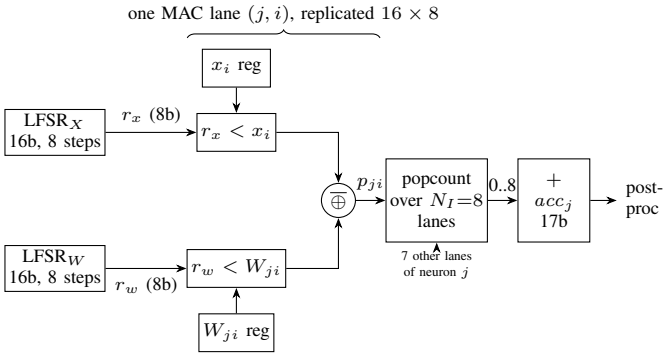


Fig. 2. One MAC lane of the SC tile and the APC of neuron j. The input comparator $[r_x < x_i]$ is shared by the 16 neuron lanes; every weight has its own comparator. In RNG mode 1 lane i uses $\mathrm{rotl}_8(r, i)$; in mode 2 each input lane has its own LFSR pair.

k, every neuron lane updates its APC accumulator

$$acc_j \leftarrow acc_j + \sum_{i=0}^{N_I-1} \overline{[r_x(i,k) < x_i] \oplus [r_w(j,i,k) < W_{ji}]}. \tag{2}$$

The accumulators are cleared once per layer and keep running over its tiles, so after the last tile acc_j is the count of ones of the complete dot product.

D. Leap-forward LFSR and RNG modes

All generators are 16-bit Fibonacci LFSRs with the polynomial $x^{16} + x^{14} + x^{13} + x^{11} + 1$ (period 65535, maximal, CALCULATION). To avoid the shifted-copy correlation of consecutive single-step outputs, each stream cycle advances every LFSR by *eight* steps and uses the low byte of the new state as r (ADR-005). Symbolic expansion over GF(2) shows that each of the eight new bits is the XOR of exactly four current-state bits while the other eight bits are a plain shift, so the leap costs eight LUT4s and one logic level per LFSR (CALCULATION). The golden model verifies the unrolled network against eight single steps for all 65536 states.

Three RNG modes are a synthesis parameter (ADR-003):

- *Mode 0 (shared)*: one LFSR$_X$ (seed `0xACE1`) for all input lanes and one LFSR$_W$ (seed `0x7A5C`) for all weight lanes.
- *Mode 1 (rotate)*: the same two LFSRs, lane i uses $\mathrm{rotl}_8(r, i)$, a free re-wiring.
- *Mode 2 (independent)*: one LFSR pair per input lane, seeds $(\texttt{0xACE1} + \texttt{0x3C5A}\, i) \bmod 2^{16}$ and $(\texttt{0x7A5C} + \texttt{0x2B67}\, i) \bmod 2^{16}$, shared by the 16 neuron lanes.

The LFSRs are loaded with their seeds when START is accepted, advance only in stream cycles and are not reset between tiles or layers. In a pre-RTL calibration benchmark of one tile, mode 2 cost 112 LUT4 and 224 flip-flops more than modes 0/1 (SIMULATION of synthesis and place-and-route). The rev A default is mode 1. A design revision (rev B) that makes mode 2 the default and re-spaces its seeds is in progress; its seeds are **[TBD:revB-seeds]**.

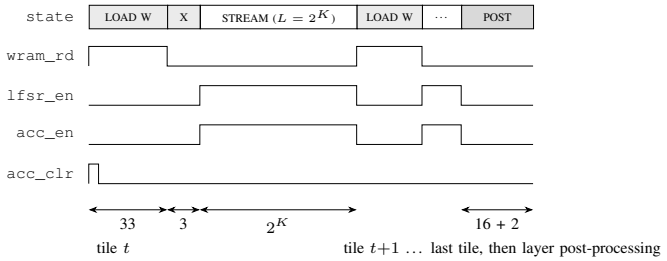


Fig. 3. Tile schedule of the latency budget (durations in clock cycles, ASSUMPTION: $O_{\text{tile}} = 32{+}1{+}3 = 36$ load cycles per tile, $16+2$ cycles per layer for post-processing and clear). The LFSRs advance only while `lfsr_en` is high. Not to scale; RTL schedule **[TBD:tpu-tile-schedule]**.

E. Post-processing (bit-exact)

After the last tile of a layer each neuron j is post-processed with integer arithmetic exactly as in the specification (rev A, Sec. 5.3), with N_{in} the layer fan-in, $K \in \{4, \ldots, 10\}$ and the activation shift $s \in \{0, \ldots, 7\}$:

$$D_j = 2\, acc_j - N_{in} L, \tag{3}$$

$$B_j = \left\lfloor bias_j\, 2^K / 128 \right\rfloor, \qquad D'_j = D_j + B_j, \tag{4}$$

$$u_j = \left\lfloor D'_j\, 2^{s+7} / 2^K \right\rfloor,$$

$$\mathrm{HIDDEN}_j = 128 + \mathrm{clip}(u_j, 0, 127), \tag{5}$$

$$\mathrm{SCORE}_j = D'_j \text{ (signed, 32 bits)}, \tag{6}$$

where (5) applies to layer 1 (hidden codes), (6) to layer 2, $bias_j$ is a signed 16-bit integer in units of $1/128$, and $\mathrm{RESULT} = \arg\max_{j \in \{0..9\}} \mathrm{SCORE}_j$ with ties resolved to the lowest index. Since $E[\overline{a \oplus b}] = (1 + v_a v_b)/2$, D'_j/L is an unbiased estimate of $\sum_i v(x_i) v(W_{ji}) + bias_j/128$ up to the floor in B_j. The accumulator needs 17 unsigned bits and D' 20 signed bits; the product $D'\, 2^{s+7}$ reaches about $2^{32.3}$ and is therefore implemented as a saturating shift rather than a multiplication (CALCULATION).

F. Programming model and tile schedule

Weights and biases are written once through the bus into the weight RAM and the BIAS buffer. Per inference the CPU writes the 64 input codes (16 words), sets LOG2_L and ACT_SHIFT in CONFIG, writes START, polls STATUS.DONE and reads RESULT; the CYCLES register reports the busy time. Fig. 3 shows the schedule of a tile as assumed by the latency budget: 32 word reads from the weight RAM, one cycle of BSRAM latency and three cycles to load the inputs, then L stream cycles; after the last tile of a layer, 16 cycles of post-processing (one neuron per cycle) and 2 cycles of clear/drain. The cycle-accurate schedule of the RTL is **[TBD:tpu-tile-schedule]**.

IV. Co-Design and Verification Methodology

A. Bit-exact golden model

The Python golden model implements the specification twice: a literal reference with explicit loops and eight single LFSR steps per stream cycle, and a vectorised engine that

pre-computes, per tile and lane, a count table from the histogram of (r_x, r_w) pairs, which makes the 360-image accuracy sweeps fast. The two agree on 16 random cases per mode for $K = 4..10$ including extreme codes and biases, and the exported test vectors (257 vectors $\times$ 33 fields: hidden codes, scores, result) were regenerated by the reference model with 0 mismatches (SIMULATION). For the RTL testbench the model exports 360 vectors per RNG mode at $K = 8$, additional vectors at $K = 4$ and $K = 10$, corner-case parameter sets and cycle-by-cycle RNG traces. The RTL-versus-model comparison is **[TBD:tpu-bitexact-mismatches]** mismatches over **[TBD:tpu-vectors-total]** vectors.

B. Training with SC noise

The data set is the 8×8 handwritten-digits set distributed with scikit-learn [12], [13] (1797 images, pixels 0..16). A stratified split gives 1437 training and 360 test images; 288 of the training images form a validation split used for every model choice, and the test split is used only for the final numbers. Pixels map to codes $c = 128 + \lfloor (127p + 8)/16 \rfloor$, i.e. non-negative bipolar values. Three variants were trained (Adam, 300 epochs, straight-through estimator for the code rounding): *float* (rounded after training), *qat* (quantisation-aware) and *qat_noise*, which adds zero-mean Gaussian noise with the independent-stream variance of (8) at $K = 8$ to both layers' pre-activations during training. The selected model is qat_noise with $s = 0$, chosen on the validation split.

C. Analytical error model

Writing each stream bit as a spin $\sigma = 2b - 1$ and assuming uniform, cycle-independent r and independent X and W generators, the variance of the estimate D'/L of one neuron is (CALCULATION)

$$\mathrm{Var}\left(\frac{D'}{L}\right) = \frac{1}{L}\sum_{\text{tiles}}\Big[\sum_{i,k} C^x_{ik}C^w_{ik} - \Big(\sum_i v_{x,i}v_{w,i}\Big)^2\Big], \quad (7)$$

where C_{ik} is the spin correlation of lanes i and k within a tile ($C_{ii} = 1$). With shared numbers (mode 0) $C_{ik} = 1 - 2|a - b|/256$ for codes a, b; mode 1 is enumerated exactly over the 256 values of r; for independent lanes (mode 2) it reduces to

$$\mathrm{Var}(D'/L) = \frac{1}{L}\sum_i \left(1 - v^2_{x,i}v^2_{w,i}\right). \quad (8)$$

Two consequences follow. The variance is largest for zero products; since most digit pixels are blank (code 128), layer 1 operates close to its worst case $64/L$. Sharing random numbers makes lanes with similar codes almost perfectly correlated and can multiply the variance by up to $N_I = 8$.

D. CPU verification by ISS-based differential testing

CPU stimuli are produced by a Python reference assembler and checked on a Python instruction-set simulator (ISS); they are testbench stimuli, not firmware. The suite has 44 programs: 11 directed, 20 random (200-instruction bodies, recorded seeds) and 13 trap tests covering every trap cause. Each program ends by dumping all registers into a 32-word signature. The ISS and assembler are covered by 93 unit tests, including hand-derived encodings and an assemble/disassemble round trip over 48 502 seeded instruction words with 0 mismatches. The self-checking RTL testbench runs every program under four bus timings (0, 1 and 3 wait states, and random 0–7 cycle stalls) and compares the signature, stop kind, trap cause, halt PC, retired-instruction count, TOHOST value, bus-error flag, LED and UART output with the ISS. A bus-protocol checker enforces seven rules (stable requests while waiting, no ready without valid, no X/Z, aligned fetches, exact byte strobes, one data access per instruction, silence after halt).

E. Fault injection

A passing testbench proves little unless it can fail. Every testbench is therefore re-run with an injected fault and must report failure: for the CPU, a corrupted expected signature and a request that changes while waiting for `mem_ready` both produce `TEST FAILED` with a non-zero exit code; the test generator itself is checked by a deliberately wrong expected value. The SC-TPU testbench fault-injection result is **[TBD:tpu-tb-faultinj]**.

F. Open-source implementation flow

Synthesis uses Yosys 0.70 with the Gowin script and wide-function multiplexers disabled (option `-nowidelut`); placement and routing use nextpnr-0.11.1 (himbaechel back end, family GW1N-9C, 27 MHz constraint), and the bitstream is packed with Apicula 0.32 only as a proof of routability; it was never programmed into a device. Simulation uses Icarus Verilog 12.0 [14] and lint Verilator 5.020 [15]. Each design is placed and routed with three seeds and the *worst* post-route Fmax is reported. The acceptance targets are Fmax $\geq$ 29.7 MHz (27 MHz + 10 %) and at most 85 % of each device resource.

V. Results

All results in this section are SIMULATION (golden model, RTL simulation or static timing analysis of the place-and-route result) or CALCULATION; none is a hardware measurement.

A. Accuracy versus stream length and RNG mode

Fig. 4 and Table II give the test accuracy of the bit-exact model with the rev A hardware seeds and, as a check that one seed set is not a lucky or unlucky draw, the mean over 16 analysis-only LFSR start offsets. The reference is the expected-value (EV) model, in which every stream is replaced by its expectation: 95.0 % (342/360, CI 92.2–96.8 %). An exact int8 binary datapath on the same codes gives the identical result (0 mismatches in hidden codes, scores and results), so it serves as the non-SC baseline. For comparison, a float-trained network evaluated in floating point reaches 96.1 % (346/360).

Three observations follow. First, decorrelation matters more than stream length at moderate K: at the reset default $K = 8$ ($L = 256$) the hardware seeds give 65.6 % with shared numbers, 91.1 % with rotation and 94.4 % with independent generators. Second, the smallest K at which the hardware-seed accuracy stays within 1 percentage point (pp) of EV

TABLE II

TEST ACCURACY [%] VERSUS K (SIMULATION, GOLDEN MODEL, SELECTED MODEL QAT_NOISE, $s = 0$, $n = 360$). HW: REV A HARDWARE SEEDS, CORRECT COUNT k AND 95 % WILSON CI; ENS.: MEAN ± S.D. OVER 16 LFSR START OFFSETS. EV ($K \to \infty$): 95.0 (342/360, CI 92.2–96.8).

K	Mode 0 (shared) HW (k)	Mode 0 (shared) CI	Mode 0 (shared) Ens.	Mode 1 (rotate) HW (k)	Mode 1 (rotate) CI	Mode 1 (rotate) Ens.	Mode 2 (independent) HW (k)	Mode 2 (independent) CI	Mode 2 (independent) Ens.
4	58.6 (211)	53.5–63.6	41.9±17.6	63.6 (229)	58.5–68.4	56.3±11.7	45.3 (163)	40.2–50.4	59.6±13.9
5	41.4 (149)	36.4–46.5	54.4±20.1	66.4 (239)	61.4–71.1	68.7±10.7	75.3 (271)	70.6–79.5	70.8±10.2
6	65.3 (235)	60.2–70.0	65.2±18.3	84.2 (303)	80.0–87.6	86.5±4.6	89.2 (321)	85.5–92.0	84.5±4.5
7	49.7 (179)	44.6–54.9	81.6±13.4	92.2 (332)	89.0–94.6	90.1±2.6	88.9 (320)	85.2–91.7	90.5±1.6
8	65.6 (236)	60.5–70.3	89.7±6.2	91.1 (328)	87.7–93.6	93.3±1.9	94.4 (340)	91.6–96.4	92.7±1.5
9	88.6 (319)	84.9–91.5	93.7±1.2	94.7 (341)	91.9–96.6	94.4±1.0	94.4 (340)	91.6–96.4	94.4±0.6
10	94.4 (340)	91.6–96.4	94.5±0.8	94.7 (341)	91.9–96.6	95.1±0.6	95.3 (343)	92.6–97.0	94.8±0.5

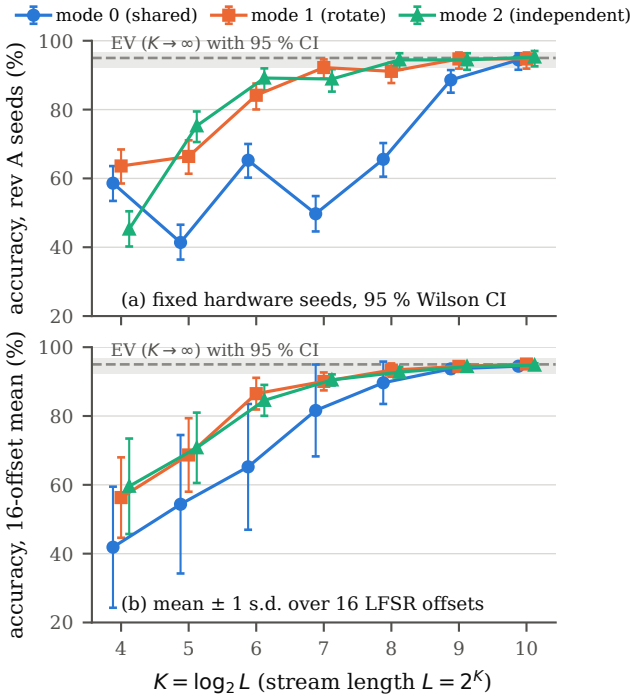


Fig. 4. Test accuracy ($n = 360$) of the bit-exact model versus K for the three RNG modes (SIMULATION). (a) Rev A hardware seeds with 95 % Wilson intervals; (b) mean ± one standard deviation over 16 LFSR start offsets. Grey: EV model with its 95 % interval. Data: `reports/sw/accuracy.json`.

for all larger K is 10, 9 and 8 for modes 0, 1 and 2; with the ensemble mean it is 10, 9 and 9. Third, mode 0 is non-monotonic in K (49.7 % at $K = 7$, 65.6 % at $K = 8$) although its ensemble mean rises monotonically (81.6 % and 89.7 %): with one shared generator pair all images see the same few empirical (r_x, r_w) histograms, so one seed set has few degrees of freedom. Note that 1 pp corresponds to 3.6 images, well below the 95 % CI half-width of 2.3–3.1 pp for accuracies between 95 and 90 %; in particular the 2-image gap between mode 2 at $K = 8$ (340/360) and EV (342/360) is not statistically resolved. Noise-aware training did not change the $s = 0$ result significantly (mode 2, $K = 8$: 95.3 % for qat versus 94.4 % for qat_noise), but post-training rounding of a float-trained model did not reach EV−1 pp for any $K \leq 10$. The rev B seeds will change the mode-2 hardware-seed row; its $K = 8$ value is **[TBD:acc-revB-mode2-K8]**.

B. Error model versus simulation

Fig. 5 compares the RMS error of D'/L (deterministic bias term removed) with (7). Over the 16-offset ensemble the ratio of measured to predicted RMS lies within 0.82–1.13 for 41 of the 42 layer/K/mode combinations; the exception is mode 0, layer 1, $K = 10$ (0.74). The prediction tracks the $1/\sqrt{L}$ law and the penalty of mode 0 (at $K = 8$, layer 1: 0.82 measured, 0.79 predicted, versus 0.50/0.48 for mode 2). A single hardware-seed realisation can deviate much more: 1.55 for mode 0 at $K = 8$, a factor 1.96 above the prediction, which explains the mode-0 accuracy dips. The model treats different stream cycles as independent. All LFSRs, however, run on the same maximal-length sequence: with the 8-step leap, two generators whose seeds are d single steps apart produce the same numbers with a lag of $d \cdot 8^{-1} \bmod 65535$ stream cycles. In rev A mode 2 the smallest such lag is 321 cycles (X_0/W_3), i.e. inside one layer from $K = 6$; the ensemble agreement for mode 2 (0.89–1.06) suggests a small effect. At $K = 10$ an inference uses 1.25 LFSR periods, so the sequence wraps within one inference (CALCULATION).

C. Latency and throughput

With the tile schedule of Fig. 3, an inference takes (CALCULATION, inputs partly ASSUMPTION)

$$N(K) = 10 \cdot 2^K + 409 \text{ clock cycles}, \tag{9}$$

i.e. ten tiles of 2^K stream cycles plus 360 cycles of tile loading and 49 cycles of post-processing, argmax and hand-shaking. Table III pairs the latency with the accuracy of Table II. At $K = 8$ an inference takes 2969 cycles or 110 µs at 27 MHz (9094 inferences/s), of which 86.2 % are stream cycles. The measured CYCLES register value of the RTL is **[TBD:tpu-cycles-K8]**; the requirement allows at most $1.10\,N(K)$. The CPU-side overhead per inference (writing inputs, START, polling, reading RESULT) is **[TBD:soc-cpu-overhead-cycles]**.

D. CPU results

All 44 programs pass under all four bus timings (176/176 runs) with zero protocol violations, and both injected faults are

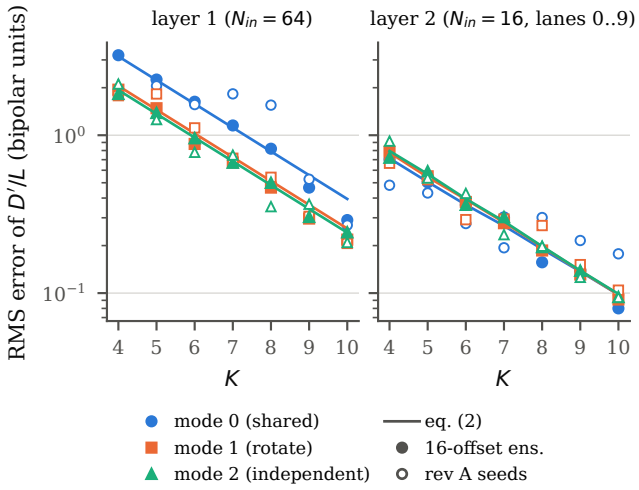


Fig. 5. RMS stochastic error of D'/L per neuron versus K (SIMULATION and CALCULATION): ensemble measurement (filled), prediction (7) (lines) and the single rev A hardware-seed realisation (open). Layer 2 uses the SC hidden codes of the same run.

TABLE III
LATENCY $N(K)$ (CALCULATION, EQ. (9)) AT 27 MHZ AND ACCURACY WITH REV A HARDWARE SEEDS (SIMULATION)

K	L	cycles	time [µs]	inf./s	acc. m1 / m2 [%]
4	16	569	21.07	47 452	63.6 / 45.3
6	64	1049	38.85	25 739	84.2 / 89.2
8	256	2969	109.96	9 094	91.1 / 94.4
10	1024	10649	394.41	2 535	94.7 / 95.3
RTL CYCLES at $K = 8$: **[TBD:tpu-cycles-K8]**					

detected (SIMULATION, Icarus Verilog 12.0). The aggregate CPI over the suite is 4.231 at zero wait states (28 206 cycles for 6667 instructions), 5.593 with one wait state (the block-RAM slave of the SoC), 8.317 with three and 9.013 with random stalls. Placed and routed with the 4 KiB RAM, the core closes at 62.46, 53.26 and 55.74 MHz for seeds 1–3, i.e. 53.26 MHz worst case, 79 % above the 29.7 MHz acceptance target (SIMULATION, static timing analysis). The worst path (18.77 ns, 9.66 ns logic and 9.11 ns routing) runs from the instruction register through the decoder to the FSM next-state logic. The core alone maps to 831 LUTs, 74 carry (ALU) cells, 111 flip-flops and two block RAMs; with the RAM and wrapper it occupies 957 of 8640 LUT4 positions (11 %) and 4 of 26 block RAMs. A first version with a barrel shifter needed 1212 LUT4 positions plus 463 wide-function multiplexers in the same context, which motivated the iterative shifter.

E. Resources and timing

Table IV collects the implementation figures. Before the RTL existed, a calibration benchmark of one 16×8 tile (weight and input registers, LFSRs, comparators, XNOR, popcount and 17-bit accumulators) was synthesised and routed to size the design. It needs 2855 LUT4 positions in modes 0/1, of which 1758 are carry cells, mostly the 136 eight-bit SNG comparators; this is about 22 LUT4 per MAC lane, 2.4 times the first estimate. The bottom-up SoC budget built on it is 5735 LUT4 (66.4 %) and 2489 flip-flops (38.4 %) (CALCULATION). The figures of the actual SC-TPU and of the integrated SoC are pending.

VI. DISCUSSION AND LIMITATIONS

SC versus binary arithmetic on this device. The SC tile provides 128 bit-level MAC lanes without using any of the 20 hard multipliers, and its precision is chosen at run time through CONFIG. Its throughput, however, is low: at $K = 8$ the calculated rate is 1.08×10^7 useful MAC/s, whereas the 40 9×9 multipliers of the device clocked at 27 MHz could in principle deliver 1.08×10^9 int8 MAC/s (CALCULATION, no such design was built). On this device SC is therefore a study of area, decorrelation and verification, not a throughput win; its appeal grows where hard multipliers are absent or energy per operation dominates, which we have not measured.

Simulation only. No bitstream has been loaded into a device and no quantity in this paper is a hardware measurement. Fmax values come from the static timing model of nextpnr, whose Gowin delay data are reverse-engineered and not validated against silicon. No power figure is reported: no open power model exists for the GW1N family, and the vendor estimator is outside the open-source flow. A physical validation plan is part of the project requirements but has not been executed.

Statistical resolution. With 360 test images one image is 0.28 pp and the 95 % interval half-width is 2.3–3.1 pp for accuracies between 95 and 90 %. Differences of 1–2 pp between modes or between $K = 8$ and $K = 9$ are not resolved, and the selected operating points should be read as engineering choices. The seed ensemble uses 16 start offsets; the project requirement asks for at least 20 (**[TBD:ensemble-20-seeds]**).

Seeding. The accuracy of a fixed seed set differs from the ensemble mean by up to 24 pp in mode 0 at $K = 8$ and the LFSR lag analysis shows that seed spacing must be judged in stream cycles after the leap, not in single steps. Rev B addresses this for mode 2; its effect is **[TBD:acc-revB-mode2-K8]**.

Workload and data sources. The 64-16-10 MLP on 8×8 digits is a deliberately tiny workload; conclusions about accuracy versus K may not transfer to larger networks. The Gowin data sheet and memory user guide could not be opened during this work; device totals were taken from the board vendor's documentation and the open-source chip database, which agree, while the block-RAM organisation remains an assumption confirmed only by successful synthesis.

VII. CONCLUSION

We have specified, modelled and partly implemented a stochastic-computing tensor unit and an RV32I processor for a 9K-LUT FPGA with a fully open-source flow. Treating the random-number generators as part of a bit-exact contract makes the SC datapath verifiable against a golden model and

TABLE IV
RESOURCES (GW1NR-9: 8640 LUT4, 6480 FF, 26 BSRAM) AND WORST-SEED FMAX. LUT4 COUNTS INCLUDE CARRY CELLS AND LUT-RAM POSITIONS.

Block	LUT4	FF	BSRAM	Fmax worst seed [MHz]	Label
CPU + 4 KiB RAM	957	126	4	53.26	SIMULATION (P&R, 3 seeds)
SC tile bench, m0/m1	2855	1454	0	52.12	SIMULATION (P&R of calibration bench)
SC tile bench, m2	2967	1678	0	52.98	SIMULATION (P&R of calibration bench)
SoC budget (m1)	5735	2489	3	–	CALCULATION (bottom-up budget)
SC-TPU (RTL)	**[TBD:tpu-lut]**	**[TBD:tpu-ff]**	**[TBD:tpu-bsram]**	**[TBD:tpu-fmax]**	pending (RTL in progress)
SoC (RTL)	**[TBD:soc-lut]**	**[TBD:soc-ff]**	**[TBD:soc-bsram]**	**[TBD:soc-fmax-worst]**	pending

exposes the role of decorrelation: in simulation, independent per-lane generators (rev A seeds) reach 94.4 % at $L = 256$, within the confidence interval of the 95.0 % expected-value accuracy, while shared generators fall to 65.6 %. A closed-form variance model predicts the stochastic error within a factor 0.74–1.13 and gives a design rule for the stream length. The CPU is verified against an ISS on 176 program runs and closes timing with a wide margin. Pending items are the SC-TPU and SoC implementation results (**[TBD:tpu-lut]**, **[TBD:soc-fmax-worst]**), the RTL bit-exactness check (**[TBD:tpu-bitexact-mismatches]**), the rev B seed study and, eventually, measurements on hardware.

REFERENCES

[1] Sipeed, "Tang Nano 9K," Sipeed Wiki (source file https://github.com/sipeed/sipeed_wiki, docs/hardware/en/tang/Tang-Nano-9K/Nano-9K.md), accessed 2026-11-04.

[2] B. R. Gaines, "Stochastic computing systems," in *Advances in Information Systems Science*, J. T. Tou, Ed. New York, NY, USA: Plenum Press, 1969, vol. 2.

[3] A. Alaghi and J. P. Hayes, "Survey of stochastic computing," *ACM Transactions on Embedded Computing Systems*, vol. 12, no. 2s, 2013.

[4] B. D. Brown and H. C. Card, "Stochastic neural computation I: Computational elements," *IEEE Transactions on Computers*, vol. 50, no. 9, 2001.

[5] K. Kim *et al.*, "Dynamic energy-accuracy trade-off using stochastic computing in deep neural networks," in *Proceedings of the 53rd Annual Design Automation Conference (DAC)*, 2016.

[6] A. Ren *et al.*, "SC-DCNN: Highly-scalable deep convolutional neural network using stochastic computing," in *Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2017.

[7] A. Waterman and K. Asanović, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, RISC-V Foundation, 2019, document version 20191213.

[8] D. Shah *et al.*, "Yosys+nextpnr: An open source framework from Verilog to bitstream for commercial FPGAs," in *Proceedings of the IEEE International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2019.

[9] C. Wolf, "Yosys open SYnthesis suite," https://github.com/YosysHQ/yosys, version 0.70 (YoWASP build) used in this work.

[10] "Project Apicula: open-source bitstream documentation and tools for Gowin FPGAs," https://github.com/YosysHQ/apicula, version 0.32 used in this work.

[11] C. Wolf, "PicoRV32 – a size-optimized RISC-V CPU," GitHub repository, https://github.com/YosysHQ/picorv32, commit ef203c2 used as a calibration proxy.

[12] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[13] E. Alpaydin and C. Kaynak, "Optical recognition of handwritten digits," UCI Machine Learning Repository, 1998.

[14] S. Williams, "Icarus Verilog," https://github.com/steveicarus/iverilog, version 12.0 used in this work.

[15] W. Snyder, "Verilator," https://www.veripool.org/verilator/, version 5.020 used in this work.